Executive Summary

CloudServe’s brief describes a support operation with more than 500 tickets per week, first replies taking 8–12 hours against a two-hour promise, first-contact resolution of 42%, and CSAT of 3.2/5. The capstone implements a local, deterministic support-automation baseline with ingestion, classification, retrieval, routing, response drafting, blocking guardrails, decision logging, and an unattended evaluation harness.

The validation run processed 80 tickets and the development run processed 500 tickets. These are local supplied datasets, not the hidden assessment set. The implementation demonstrates the end-to-end pipeline and logging mechanics, but its heuristic classifier is not production-grade. The auto-response percentage is not proof of resolution uplift.

1. The Problem

CloudServe needs faster support without creating new risks through incorrect or overconfident automated answers. An automation system must distinguish documentation-answerable questions from ambiguous, sensitive, high-urgency, or unsupported requests. A high automation rate is not a useful outcome if customers receive unsupported instructions.

The brief-provided baseline values are used as context only. The project has no live customer deployment, no actual first-reply timestamps produced by a production service, and no direct CSAT survey for the new system.

2. Discovery Evidence and Constraints

The supplied pack contains 500 labelled development tickets, 80 validation tickets, 29 knowledge-base documents, 200 reference responses and five stakeholder interview transcripts. The interview transcripts are primary discovery evidence and must be read and cited by the submitting student before making qualitative claims.

To avoid inventing findings, the included discovery workbook is marked as a draft and explicitly asks the author to add transcript-grounded observations. This is a remaining personal evidence task, not a result that can be inferred from the brief.

3. Requirements and Traceability

FR-01 normalizes email, chat, documentation comments and forum tickets. FR-02 assigns intent, urgency, confidence and alternatives. FR-03 retrieves ranked document passages with resolvable IDs and a relevance threshold. FR-04 applies deterministic routing. FR-05 drafts cited responses and abstains when evidence is missing. FR-06 blocks private data, invalid citations and unsupported commitments. FR-07 logs every decision. FR-08 runs unattended over arbitrary input paths and emits metrics. FR-09 exposes a FastAPI health and ticket endpoint.

Traceability is maintained in docs/architecture.md, with implementation paths and tests linked to each functional requirement.

4. Architecture Overview

The pipeline is: input JSON → normalize → classify → retrieve → route → draft → validate → auto-response or escalation → JSONL decision log → metrics. Each component is isolated in src/ so that its behaviour can be tested separately.

The baseline uses a deterministic local approach instead of an external LLM provider. This removes API keys, rate limits and provider outages from the execution path, while sacrificing natural-language reasoning and answer quality. The choice is suitable for reproducibility, not a claim that the local baseline is production-ready.

5. Ingestion and Normalization

src/ingest.py maps input fields to a stable internal representation. It preserves the original text, source channel and original payload, fills missing fields with safe defaults, supports channel aliases, and generates a stable fallback ticket identifier when one is missing.

The test suite covers all four expected channels and empty-body fallback. In a production service, schema validation should also enforce payload-size limits, strict timestamp parsing and a documented data-retention policy.

6. Classification

The classifier uses labelled development examples and a pure-Python token-overlap nearest-neighbour method. It returns an intent, urgency, confidence proxy, alternatives and a method label. A fallback handles empty or unclassifiable content.

The confidence score is heuristic, not calibrated probability. The routing threshold of 0.68 must not be interpreted as a statistical guarantee. Evaluation shows label-taxonomy mismatches and rare-class weakness; a production iteration should use a trained classifier, normalized label taxonomy, calibration curves and per-class threshold selection.

7. Retrieval and Grounding

src/retrieve.py ranks the supplied documentation using IDF-weighted lexical overlap and returns document IDs, scores and passage text. Results below a relevance threshold are omitted. Source IDs are drawn from the real 29-document corpus, enabling traceability.

Lexical retrieval is transparent and inexpensive, but it can miss semantically similar paraphrases, misspellings and non-fluent English. A later version should compare a lexical baseline with embeddings on a held-out relevance set and use human-labelled retrieval relevance rather than hit rate alone.

8. Routing and Response Generation

The route is deterministic: critical requests, empty tickets, selected sensitive topics, low-confidence classifications and missing retrieval evidence are escalated. A generated response is only eligible for release after validation. Escalations carry a ticket summary and retrieved source identifiers in the decision record.

The response generator extracts a short fragment from the highest-ranked passage and adds a citation. It does not call a language model and does not assert that an account action was performed. This is deliberately conservative but can produce awkward or incomplete answers.

9. Guardrails and Governance

The validator checks for email addresses, phone-like patterns, API-secret patterns, unsupported commitments and citations not present in the retrieved source list. A blocking finding prevents release. Every decision records the guardrail status and findings.

Pattern-based detection is incomplete and can produce false positives. Production use requires broader sensitive-data detection, adversarial testing, access-controlled logs, retention rules, human review and a remote kill switch. The local immediate stop mechanism is process shutdown; the remote switch remains an open production requirement.

10. Evaluation Method

The harness accepts input and output paths, processes each ticket without interactive intervention, isolates per-ticket exceptions, writes JSONL results and decision logs, and generates a metrics JSON file. It was run once over the supplied 80-ticket validation file and once over the 500-ticket development file for debugging. The development-set figures are not holdout results because development labels informed the nearest-neighbour classifier.

The hidden assessment dataset was not available to this run and was not used. The assessment report must be updated if and when the hidden set is run by the assessor.

Measure	Value
Processed	80
Auto-response	42
Escalated	38
Blocked	0

11. Validation Run — Volume and Business Proxies

Validation tickets processed: 80. Auto-response decisions: 42. Escalations: 38. Guardrail blocks: 0.

Validation auto-response share: 52.5%. Escalation rate: 47.5%. Historical FCR recorded in the validation tickets: 46.25%. Historical CSAT proxy mean: 2.6.

These metrics describe actions and historical ticket metadata. They do not show that the new system achieved the 60% FCR target or improved CSAT. The harness does not fabricate response-time values when real customer timestamps are unavailable.

Technical measure	Value
Macro precision	0.1562
Macro recall	0.2311
Retrieval hit rate	100.0%
Latency p95	0.303 ms

12. Validation Run — Technical Metrics

Macro intent precision: 0.1562; macro recall: 0.2311. Retrieval hit rate: 100.0%. Pipeline median latency: 0.231 ms; p95 latency: 0.303 ms.

The per-class precision/recall table and confusion matrix are available in evaluation/results/validation_tickets_metrics.json. Several classes have zero precision or recall because the heuristic taxonomy does not align well with the full label set. This is a clear limitation, not a result to hide.

Governance measure	Value
Decisions logged	80
Reconciles	True
PII detections	0

13. Validation Run — Governance Metrics

Decision log records: 80; log-to-ticket reconciliation: True. Private-data detections: 0. Guardrail activations by type: {}.

Zero detections in this run do not prove that all sensitive data would be caught. The test suite explicitly verifies that a sample email pattern and an invalid citation block output. A fuller evaluation should include multiple PII types, secrets, prompt injection and unsafe commitment test cases.

14. Fairness and Segment Review

The data schema supports segmentation by customer tier, region, language fluency, channel and text length. A valid fairness audit must compare response quality and routing outcomes per segment, report group sample sizes, and flag gaps above five percentage points. Small groups should be marked insufficient sample.

This package documents the audit method but does not claim a completed human-scored fairness audit. Lexical matching may disadvantage non-fluent English and paraphrased requests; this risk is in docs/governance.md and must be checked against the ticket data before deployment.

15. Incident Response and Kill Switch

The incident procedure is detect, contain, assess, notify, remediate and review. The immediate containment step is to stop the API process and switch support handling to manual review. Production operations need a remote feature flag that disables automatic sending without a redeploy, plus alerts on block spikes, latency, and log-reconciliation failures.

No system should continue automatically responding if decision logs stop reconciling, a PII incident is suspected, or the safety validator is unavailable.

16. CI, Testing and Reproducibility

The repository includes a GitHub Actions workflow that installs dependencies and runs pytest. Seven local tests pass in the build environment. The README documents virtual-environment setup, the single test command, the API start command and the unattended evaluation command.

A clean-checkout rehearsal on a separate machine remains recommended before submission. The API dependencies should be installed as documented. The offline harness itself uses the Python standard library and does not require model credentials.

17. Requirements Revision

The revision log records three substantive changes after implementation: nearest-neighbour classification from labelled development examples, per-ticket exception isolation with logged escalation fallback, and explicit separation of auto-response share from actual first-contact resolution.

The next revision should be driven by the measured validation confusion matrix and evidence from the interview transcripts. Any tuning should use development data; the hidden assessment set must not be used for iterative optimization.

18. Limitations and Next Steps

The figures above should be treated with caution because classifier confidence is a heuristic, lexical overlap is not semantic understanding, the generator uses excerpts rather than an LLM, and historical ticket outcomes do not establish causal uplift. A human review of at least 50 answers by two independent assessors is still required to estimate hallucination rate and citation accuracy.

Next steps: read and cite all interview transcripts; finalize the problem statement and five workbooks; calibrate classification confidence; normalize the intent taxonomy; measure citation support with human raters; test fairness by segment; add a remote kill switch; rehearse clean setup; record the required video.

19. AI Assistance and Authorship Disclosure

ChatGPT assisted with implementation scaffolding, debugging and documentation drafting. The author must review and understand the submitted code and replace any statements that do not reflect their own actual work. The discovery findings and personal reflection must come from source evidence and personal experience; they must not be invented by a model.

The repository is designed to be inspectable and reproducible. The generated code is attributed here in accordance with the project rules. The submitting student should add a truthful account of any other tools used and verify all final claims.

20. Conclusion

The capstone provides an executable end-to-end baseline: all 80 validation tickets and 500 development tickets were processed, each run generated metrics and decision logs, and the seven included tests passed. The local deterministic design prioritizes reproducibility and safe abstention over high automation.

This is not a production-ready support agent. Validation classification metrics reveal material weaknesses, and human-rated grounding, hallucination, fairness and real customer outcomes remain unmeasured. The final submission still requires the student's own video, evidence-based discovery findings, truthful effort log and review of the complete repository.